ChatGPT

SvelteKit WebUI in Gradio Apps

Gradio is primarily a Python server framework, whereas a SvelteKit web UI is a separate client-side app. There is no built-in way for Gradio to "convert" a SvelteKit site into its interface, but you can **embed or proxy** the SvelteKit UI within Gradio. For example, one can use a custom Gradio HTML component or an **iframe** component (like the `gradio_iframe` custom component) to load an external webpage. Such an iframe component allows you to embed any site (e.g. the SvelteKit UI) inside a Gradio interface [1] [2]. In practice you'd run the SvelteKit app on its own (serving static HTML/CSS/JS) and include an `<iframe>` in Gradio to show it. Another option is to mount multiple Gradio "sub-apps" on one FastAPI server (one could serve a Gradio page and the SvelteKit page as different routes) and use a hash-based router for navigation [3]. However, multi-page Gradio is still experimental; the Gradio team has proposed using hash-based routing for distinct pages [3]. In summary, you **cannot directly "run" the SvelteKit UI inside Gradio** except via embedding (iframe/HTML), or by managing multiple servers on the same domain. Enabling Gradio's MCP ("Model Context Protocol") support (via `launch(mcp_server=True)`) simply exposes a tool API endpoint [4] – it does *not* auto-host a SvelteKit frontend. MCP makes the Gradio app an LLM-accessible tool, but any HTML UI (Gradio or Svelte) still runs separately.

Sharing Data and State (Gradio ↔ SvelteKit)

To exchange data between a Gradio backend and a SvelteKit frontend, the simplest approach is via **HTTP APIs**. Gradio natively provides REST endpoints (e.g. `/predict`) for its interface. A SvelteKit app can call these endpoints (using `fetch`) to send inputs and receive inference outputs. Gradio even offers an official **JavaScript client** (`@gradio/client`) that lets you connect to a Gradio Space or app by name and call its endpoints [5] [6]. For example:

```
import { Client } from "@gradio/client";
const app = await Client.connect("username/space-name");  // Gradio Space or
server
const result = await app.predict("/predict", { input: "Hello" });
```

This pattern allows a SvelteKit page to drive the Gradio model remotely. Conversely, Gradio can be used purely as a tool server (for LLMs or other apps) and the SvelteKit UI can pull results. Session state (e.g. conversation history) isn't shared automatically; you'd need to manage that in your app logic (for example, passing a session ID or keeping state in the client). For live updates or streaming, one could poll the Gradio API or use `app.submit()` which returns an async iterator of "data" and "status" events [7] [8]. In practice, we've seen Svelte frontends call Gradio APIs to fetch results as needed, since Gradio has no native client sync beyond these HTTP interfaces. (Alternatively, custom components or websockets could relay data, but that requires more custom coding.)

Hugging Face Spaces Deployment (Gradio vs Svelte)

Hugging Face Spaces supports different app types. You can deploy a Gradio app by choosing the Gradio SDK, or a static web app (React/Svelte/etc.) by using `sdk: static`. **Static Spaces** let you host SvelteKit apps (with a build step) [9]. For example, set in your space's README:

```
sdk: static
app_build_command: npm run build
app_file: build/index.html
```

This will build and serve your SvelteKit frontend. Hugging Face has examples of static Svelte apps doing this [9]. Gradio apps, on the other hand, run as a Python server (choose "Gradio" SDK). A single Space can only use one SDK, so you can't directly host both in one space unless you use a **custom Docker space**. In a Docker-based Space you could run two processes (one serving the SvelteKit bundle via Nginx or similar, and one running the Gradio server), and route accordingly. Otherwise, the common strategy is to use *two* spaces: a Gradio space for the model API, and a separate static space for the SvelteKit UI, with the UI calling the Gradio space via the network (with CORS allowed). Note HF Spaces run sites in iframes on the Hub, so you should prefer a hash-based router (as Gradio discussion suggests) [3] so links work. In short: **Yes, Hugging Face Spaces can host SvelteKit UIs via static spaces** [9], but linking a Gradio app and a SvelteKit app typically means running them separately (or combining via Docker/nginx) rather than a single unified deploy.

WebGPU/WASM Local Inference Integration

Modern LLM frameworks like llama.cpp can compile to WebAssembly and use WebGPU for browser inference. For example, the wllama project provides a WASM binding for llama.cpp that runs entirely in-browser [10]. Its demo (hosted on HF Spaces) is a Svelte/Vite app that loads a quantized llama model and runs it locally with WebAssembly SIMD; **no Python backend is used**. This shows that a SvelteKit UI could perform inference on the client by loading llama.cpp in WASM/WebGPU. If you want to combine this with Gradio, one approach is hybrid: use the SvelteKit UI to do WebGPU-based inference (like wllama) and use Gradio for additional tools or for cases where heavy models need server compute. The SvelteKit code could call Gradio's API for tasks it doesn't handle locally. Alternatively, you could wrap the WASM engine in a Gradio custom component (Gradio supports embedding JS in custom HTML components) so that the Gradio app host triggers browser inference. In Hugging Face Spaces, you could run the SvelteKit+WASM app as a static space (as wllama demo does), possibly alongside a Gradio space for server-side features. Note that llama.cpp is actively adding WebGPU support in C++ (issue #7773) [11], which will further blur the line between CPU and GPU inference on device.

Examples and Compatibility Notes

- **Gradio's Svelte Components:** Gradio itself uses Svelte for its UI components, and even publishes Svelte components for integration. For example, the @gradio/dataframe is a standalone Svelte component that replicates Gradio's DataFrame widget [12]. This shows you can mix Gradio UI elements into a SvelteKit project when needed.
- **Chat UI (HuggingFace/HuggingChat):** Hugging Face's open Chat UI uses SvelteKit for frontend and llama.cpp for backend. It connects via a local llama.cpp HTTP server and runs completely independently of Gradio [13]. It proves SvelteKit can serve rich LLM interfaces (with tools, search, etc.) tied to llama.cpp. If one wanted, a Gradio backend (with `mcp_server=True`) could replace or supplement the llama.cpp server as the model API (though Chat UI isn't built for that).
- **Wllama Demo:** The wllama Space uses a WASM llama in browser; it's a straight web app (no Gradio). This illustrates the client-side option, and its code could be adapted into SvelteKit.
- **Integration Patterns:** There aren't many public examples of a single app mixing Gradio *and* SvelteKit UI. Typically, projects choose one stack (Streamlit/Gradio vs. custom web UI). If you do need both, consider the iframe embedding approach [1] [2] or the multi-app router approach

[3] . In any case, the Gradio JS client is your friend: a SvelteKit page can invoke a Gradio space's endpoints easily [5] , so the separation is technical but bridged by HTTP.

Summary: Gradio and SvelteKit can coexist, but they typically run as separate layers. You can embed a SvelteKit UI in Gradio via iframes or combined routing, and you can share data via APIs or the Gradio JS client [5] [6] . On Hugging Face, SvelteKit UIs go into static Spaces [9] , while Gradio goes into Gradio Spaces; tying them together requires custom deployment. WASM/WebGPU inference (e.g. llama.cpp in the browser) works great with SvelteKit (as in wllama [10]), and that output could be fed into a Gradio tool if needed. In practice, mix-and-match using web embeds and HTTP calls is the most practical integration strategy.

Sources: Gradio docs and guides [4] [5] [12] ; Hugging Face Spaces docs [9] ; llama.cpp and community projects [13] [10] .

---

[1] [2] GitHub - LennardZuendorf/gradio-iFrame: Custom iframe component for Gradio, empowering the existing html component.
https://github.com/LennardZuendorf/gradio-iFrame

[3] Support multiple pages in a gradio app · Issue #2654 · gradio-app/gradio · GitHub
https://github.com/gradio-app/gradio/issues/2654

[4] Building Mcp Server With Gradio
https://www.gradio.app/guides/building-mcp-server-with-gradio

[5] [6] [7] [8] Gradio js-client JS Docs
https://www.gradio.app/docs/js/js-client

[9] Static HTML Spaces
https://huggingface.co/docs/hub/en/spaces-sdks-static

[10] GitHub - ngxson/wllama: WebAssembly binding for llama.cpp - Enabling on-browser LLM inference
https://github.com/ngxson/wllama

[11] ggml : add WebGPU backend · Issue #7773 · ggml-org/llama.cpp · GitHub
https://github.com/ggml-org/llama.cpp/issues/7773

[12] Gradio dataframe JS Docs
https://www.gradio.app/docs/js/dataframe

[13] Chat UI
https://huggingface.co/docs/chat-ui/en/index